

Trabajo Práctico N° 1

Implementación de Transacciones y Reglas de Negocio

A continuación, se presentan 6 escenarios del mundo real. Para cada uno, debes escribir el bloque SQL completo (usando BEGIN, COMMIT o ROLLBACK según corresponda) que resuelva el problema garantizando las propiedades **ACID**.

Para todos los ejercicios se requiere el contenedor Docker del práctico corriendo, pgAdmin abierto (con conexión a la base de datos de la materia).

Reglas de negocio

- Los posibles estados de los pedidos son: 'PENDIENTE', 'PAGADO', 'CANCELADO'
- Cuando un cliente realiza un pedido sin pagarlo, el sistema debe registrar el pedido con estado 'PENDIENTE' y descontar inmediatamente la cantidad solicitada del stock del artículo.
- El sistema jamás debe permitir que el stock quede en negativo.
- Un cliente puede realizar un pedido y pagarlo usando su saldo a favor. En este caso el estado del pedido queda en 'PAGADO'.
- Por ley de protección de datos, un cliente puede solicitar que se elimine su cuenta. En ese caso se deben eliminar los pedidos asociados para que no queden "huerfanos".

Ejercicio 1: El cliente 1 (`id_cliente = 1`) desea comprar 5 unidades del artículo 9. Escribe una transacción que registre el pedido y descuente el stock del artículo.

Ejercicio 2: El cliente 7 (`id_cliente = 7`) desea comprar 4 unidades del artículo 3 pagando con su saldo a favor (tabla `billetera_cliente`). Escribí una transacción que registre el pedido con estado 'PAGADO', actualice el stock del artículo y descuente la plata de su billetera. ¿Qué sucede con la transacción? ¿Qué restricción de integridad bloqueó los cambios? ¿Cómo liberamos los recursos asociados a la transacción?

Ejercicio 3: El cliente 2 (`id_cliente = 2`) desea comprar 200 unidades del artículo 392 pagando con su billetera virtual del sistema. Escribí una transacción que registre el pedido con estado 'PAGADO', actualice el stock del artículo y descuente la plata de su billetera.

Ejercicio 4: El cliente 5 solicitó eliminar su cuenta. Escribí una transacción que elimine al `id_cliente = 5` del sistema sin violar la integridad referencial.

Ejercicio 5: El gerente recibió la nueva lista de precios de electrónica y nos indica que por la inflación debemos aumentar un 20% el precio de todos los artículos de la categoría 'Electrónica'. Escribí una transacción que actualice el `precio_unitario` multiplicándolo por 1.20 para los artículos de la categoría 'Electrónica'.

Ejercicio 6: Para este ejercicio, deberás abrir dos pestañas (Query Tools) diferentes en pgAdmin. Simularemos que dos vendedores intentan vender la última unidad del artículo 5 al mismo tiempo.

- Ejecuta: `UPDATE articulos SET stock = 1 WHERE id_articulo = 5;`
- En la Pestaña 1, inicia una transacción y descuenta 1 unidad del stock del artículo 5. (No hagas COMMIT todavía).
- Ve a la Pestaña 2. Sin usar BEGIN, intenta ejecutar un UPDATE que también descuenta 1 unidad del artículo 5.
Responde en un comentario dentro de tu script: ¿Qué sucede en la Pestaña 2? ¿Por qué la consulta se queda esperando indefinidamente?
- Vuelve a la Pestaña 1 y ejecuta COMMIT;.
Responde: ¿Qué pasó inmediatamente en la Pestaña 2 al hacer esto? ¿Cómo garantizó PostgreSQL la consistencia del stock?